Tokenization and Text Preprocessing

From raw documents to model-ready text

Purpose

This short reference note is written for learning, semantic retrieval, and grounded question-answering experiments. It uses original explanatory content and is organized into coherent sections rather than unrelated filler.

1. What Tokenization Does

Tokenization converts text into units a model can process. A tokenizer may split a word into one token or several subword pieces. For example, an uncommon technical term might be represented by multiple fragments rather than a single token. Token IDs are vocabulary indices, not semantic scores. The embedding layer later maps these IDs to vectors.

2. Why Subword Tokenization Is Useful

A vocabulary containing every possible word would be huge and would struggle with new names, spelling variations, and technical terms. Subword methods balance vocabulary size with coverage. Common strings can remain intact while rare words are decomposed into reusable pieces. This is especially useful for scientific writing, code, product names, and multilingual text. Different model families use different tokenization algorithms and vocabularies, so token counts can differ for the same passage.

3. Characters, Words, Tokens, and Chunks

Characters, words, model tokens, and retrieval chunks are different units. A 1,000-character passage is not necessarily a fixed number of tokens. A chunk is a segment selected by an application for storage and retrieval; it may be defined by characters, tokens, sentences, or document structure. When comparing chunking strategies, record the actual method and parameters rather than assuming that a character limit corresponds to a universal token count.

4. Preprocessing PDF Text

PDF extraction can introduce broken line endings, repeated headers, page numbers, hyphenated words, or reading-order problems. A preprocessing pipeline should normalize avoidable noise without destroying meaningful structure. For example, removing repeated page headers can improve retrieval, while deleting headings may remove valuable context. Preserve page numbers and document identifiers as metadata so retrieved passages can be traced back to their source.

5. A Practical Cleaning Checklist

First, inspect extracted text from several pages rather than assuming extraction is correct. Next, normalize whitespace and repair obvious line-wrap artifacts. Avoid aggressive global substitutions that alter code, formulas, abbreviations, or punctuation. Keep section headings near the text they describe. Finally, compare the cleaned text against the source PDF for a sample of pages. Log extraction errors because retrieval quality cannot reliably exceed the quality of the text entering the pipeline.

6. How Tokenization Affects Retrieval

Embedding models generally accept token sequences with model-specific limits. Very long input may be truncated or rejected. A chunk that is too large may exceed the embedding model’s input limit; a chunk that is too small may omit context needed to interpret a term. Token counts also affect latency and cost for hosted model APIs. The right chunk size should be tested against representative questions, not selected from a universal rule.

Key Takeaways

Tokenization is a model-specific conversion from text to token IDs. It is distinct from PDF extraction, text cleaning, and retrieval chunking. Reliable preprocessing preserves meaning, provenance, and document structure while removing accidental noise.

End of document • Use source passages as evidence and verify claims during evaluation.